

Unpacking the internal assessment

Criterion A: Problem specification

This section gives advice on the choice of problem for the computational solution and the best ways to address this criterion in the context of the overall solution.

Problem scenarios

The choice of an appropriate problem is key to the success of the computational solution. A suitable problem can come from the outside world, from a different subject in a student's DP studies, or from a computational context. The problem needs to be sufficiently complex for a solution to require use of the computational thinking process and address all the criteria. The problem should also require sufficient innovation such that the solution is demonstrably the student's own work and ideas.

For example, in developing a computer game, over and above the game having sufficient complexity, it should also have a meaningful purpose. This could be an educational game for a specific subject area or a specific target audience. The game should be original and aim to develop a specific skill set. Simply recreating an existing game, or building a new version without significant innovation, will not meet the criteria for the computational solution.

Example problem scenarios may involve:

- inventory management systems
- student databases
- library management systems
- appointment scheduling tools
- encryption/decryption utilities.

The problem can arise from any idea or need related to the students' interests. Once the problem is identified, students must articulate it clearly to construct a problem scenario. Asking some of the following questions may help in this regard.

- What is the current situation?
- What causes the problem?

Unpacking the internal assessment

- Who is affected by the problem?
- What are the objectives or general requirements for the computational solution to this problem?
- Are there any constraints associated with the problem?
- Are there existing computational solutions, and if so, why are they not effective? What are their shortcomings?
- Why is it valuable or significant to construct a computational solution for the problem?
- Are there any time limitations, resource limitations, or specific conditions?

Computational context

Once a problem scenario has been defined the student needs to decide what type of computational response is needed to address it. They will start by choosing a specific programming language or programming environment, depending on the nature of the proposed computational solution. For example, for:

- web-based solutions, use of HTML, CSS, JavaScript, etc.
- database solutions, appropriate programming languages to perform operations on data
- stand-alone solutions and mobile applications, use of C#, Java, Python, etc.

These choices need to be explained in the context of the problem scenario.

Success criteria

The success criteria are a set of measurable software specifications that will determine the effectiveness of the proposed computational solution. They are mentioned in four of the five criteria, so it is vital that students understand what is required in meeting these. The criteria must be derived from the identified problem and its requirements, and should refer to the essential functionality of the software features that will help to solve the problem.

When working on the construction of the success criteria, students must ensure:

- relevance
- clarity

Unpacking the internal assessment

- achievability
- testability.

The criteria should be concrete, easily showcased and demonstrable, align with the testing strategy outlined in the video and specified in the system overview section of the documentation.

Success criteria should not be based on a potential user's perception of the computational solution—the criteria must not be subjective (e.g. how user-friendly the solution should be or what the GUI should look like). If the UI is one of the success criteria this must be clearly stated and measurable with evidence.

Success criteria should be worded in precise terms related to the required functionalities of the computational solution.

Success criteria need to be sufficient in number (around 8–10), substantial, specific and testable.

Examples of good success criteria

- The solution will allow for the input, editing and deletion of customer records from the customer database.
- The solution will give appropriate warning messages to the user in case of extreme or invalid inputs.
- The solution will be menu-driven with eight menu choices (including an indication of the choices).

Examples of inadequate success criteria

- The code will compile.
- The code will run without crashing.
- The code will use loops.
- The program will be aesthetically pleasing.

